



Products ▾

Resources ▾

Company ▾

Pricing

Demo

All

Announcements

SBOM

Security

Licensing

Open Source

Press

[Home](#) / [Blog](#) / [Anatomy of a Software Supply Chain Attack](#)[← Back to blog](#)

Anatomy of a Software Supply Chain Attack

September 30, 2021 · 8 min read · Matthew Schwartz

cybersecurity

software security



Software supply chain attacks are exploits where a bad actor compromises a piece of software that's used by multiple targets — i.e., something in the supply chain. Software supply chain threats differ from (and have the potential to be more damaging than) classic attacks because a single breach can impact hundreds or even thousands of different targets.

Consider the hypothetical where I'm a cybercriminal and want to inject malicious code into N different targets.

In a classic attack, I'd need to come up with around N unique exploits to affect each system. While I may be able to reuse the same attack mechanism against multiple targets, there is no guarantee that will work

**FOSSA**

in every case. Even worse, it's possible that any of those N targets has relatively good security measures and is very hard to breach.

[Products](#)[Resources](#)[Company](#)[Pricing](#)

Now, consider a supply chain attack that aims to hit the same N systems. I only need to infect a single shared dependency of those N targets. What makes this even more dangerous is that I can try to find the easiest attack surface from a myriad of other sources now that I'm not constrained to only attacking my unsuspecting victims' environments directly. Now, whenever any of these N systems uses or pulls in this shared dependency, they're also importing the trojan of my choosing.

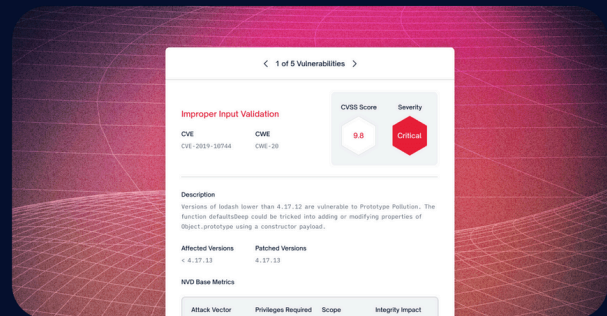
This is what makes software supply chain attacks so dangerous. One successful attack can affect a wide swath of machines.

So, what strategies can security and engineering teams employ to improve [software supply chain security](#)? What tools provide the strongest protection? Read on for answers to these questions, as well as an overview of three particularly devastating supply chain attacks.

RELATED:

[How to Prevent Dependency Confusion Attacks](#)

Get My Free Vulnerability Scan

[Request Now](#)

Supply Chain Attack Example 1: event-stream Trojan

We'll start our review of supply chain attacks with a look at the event-stream trojan. event-stream was an npm package whose maintainer no longer wanted to maintain the package, so they transferred its publishing rights. The new maintainer added a dependency called flatMap Stream, which initially was benign and worked as expected.

However, they later updated flatMap Stream to have malicious code that would sniff for bitcoin wallets with more than 100 bitcoins in them and send secret keys off to a remote server. If you updated your event-stream package or any of your dependencies and you had a version range defined for flatMap Stream, it would innocuously and unceremoniously pull in the malicious version.



Here, there was a single shared resource (the npm registry). The attackers targeted Bit Pay, but the exploit impacted other companies including Microsoft and most other unsuspecting event-stream users.

Supply Chain Attack Example 2: SolarWinds Orion Platform

The SolarWinds Orion incident is perhaps the most well-known example of a software supply chain attack. It happened because hackers got access to secure credentials — in case you're wondering, *SolarWinds123* is not a good password if your company is SolarWinds, or for that matter, any company.

Once the attackers compromised these credentials and gained access to SolarWinds' network, they started to add malicious code throughout SolarWinds' system. Specifically, the hackers targeted the Orion Platform, which is used for infrastructure monitoring and some security features for large organizations. The Orion Platform was considered a single, trusted resource. If a user downloaded from SolarWinds, they expected the payload to come from SolarWinds.

But once that payload was maliciously updated, we witnessed something similar to the event-stream attack. The SolarWinds Orion Platform was compromised, which then affected hundreds of organizations including the Federal Reserve, Microsoft, Fire Eye, and the Department of Energy. This exploit has been an ongoing thorn for well over a year at this point and still plagues many companies and the U.S. government.

Supply Chain Attack Example 3: Codecov Bash Uploader

More recently, we had the Codecov bash uploader compromise. Bad actors gained access to credentials due to an error in Codecov's **Docker image creation** process; the attackers were then able to change Codecov documentation and modify the script (which was included in the documentation) used to install the tool in CI environments.

For context, Codecov is a code coverage tool that integrates into your CI environment. You download a script to upload your code coverage results to Codecov's server. And, attackers were able to change that particular script.

Modifying the script enabled the attackers to exfiltrate secret environment variables, which is particularly bad if you deploy your production services from the same CI in which the compromised script was run. It means that all of your secrets for production should be considered compromised. To make matters worse, Twilio, GoDaddy, and most other Codecov users had no idea an attack occurred until someone checked the hash of the payload and noticed it didn't match what the hash would have been from the upload script on GitHub.

Defending Against Supply Chain Attacks

Now that we've discussed the specific tactics used in supply chain attacks, we'll shift our focus to preventing them. Broadly, we'll group these defense mechanisms into two categories: strategies and

Strategies for Defending Against Supply Chain Attacks

Use lockfiles: Lockfiles will pin the versions of every package in your dependency graph, and often they will be pinned to a specific registry. This means that even if a new version is published and your semver range specification allows your code to use it, the lockfile will not be changed unless you explicitly update it. Additionally, most lockfiles include hashes that can detect man-in-the-middle attacks where the package payload has changed.

Always try to validate checksums of any software that you use: Do this whether it involves developer tools or source code.

Vendor dependencies into your source control: Don't just install from a repository every time you want to install your dependencies. When you actually vendor in your dependencies, you're guaranteed nothing will change unless *you* change it.

Inspect all the code that's being used in production: This is obviously very, very high overhead. But there are certain domains where this is necessary. For example, I used to work at Capital One, and if you wrote code that moved money in any way, there was a team that would audit all of the code, whether it be first-party or third-party. This auditing team would take a significant amount of time and prevent production deployments until they gave the thumbs up.

Request a [software bill of materials](#) where and when possible: If practical, it's generally useful to ask suppliers to produce an SBOM to accompany any piece of software. This, of course, helps improve visibility into your software supply chain.

Consult NIST's Guidelines on Minimum Standards for Developer Verification of Software: As part of the [Biden Administration's Executive Order on Improving America's Cybersecurity](#), NIST (The U.S. government's National Institute of Standards and Technology) [published 11 recommendations for software testing](#). These include threat modeling, testing, code-based structural test cases, and more.

Tools for Defending Against Supply Chain Attacks

[Software composition analysis solutions](#) identify and provide remediation guidance for known vulnerabilities in open source code. This is an important part of security tech stacks given the popularity of OSS in modern software development — it's estimated that open source comprises 85-90% of the average application.

Google recently published its [SLSA framework](#) (Supply Chain Levels for Software Artifacts), which is meant to expose metadata in a software artifact throughout different stages of the software development lifecycle. The framework aims to ensure the "integrity of software artifacts throughout the software supply chain."

**FOSSA**[Products](#)[Resources](#)[Company](#)[Pricing](#)

A common thread in many of these strategies and tools is visibility: Oftentimes, knowing that *something* is going wrong goes a long way toward strengthening your defenses against these attacks. Going all the way back to the event-stream trojan, that particular attack was discovered because of a warning that was published on install about a soon-to-be deprecated network call on a package that wasn't supposed to be connected to the internet. That prompted someone to realize something was wonky.

The event-stream trojan, like the SolarWinds and Codecov incidents that followed it, also shows just how dangerous these attacks can be. They can be hard to detect unless you have the proper approach and infrastructure to identify anomalous behavior.

About the Author: Matt Schwartz is a team lead and senior software engineer for FOSSA, focusing on expanding and improving the company's security offerings. Outside of FOSSA, he has worked with several organizations on security and security infrastructure, including the Barbados government, and responsible disclosures to various companies.

Subscribe to our newsletter

Get the latest insights on open source license compliance and security delivered to your inbox.

Don't miss a thing.



Products

Resources

Company

Pricing

Subscribe

By submitting, I agree to receive periodic emails from FOSSA & accept the FOSSA Privacy Policy.

Platform

- Scan
- Binary
- Snippets
- Pricing

Solutions

- OSS License Compliance
- Code Security (SCA/BCA)
- SBOM Management
- Due Diligence
- Obsolescence Management
- Supplier Risk Management

Resources

- Getting Started
- Blog
- Documentation
- Resource Library
- Glossary
- Webinars

Learn

- CycloneDX
- Open Source Licenses
- SBOM Compliance Requirements
- SBOMs
- Software Composition Analysis
- Software Supply Chain Security
- SPDX

Company

- About FOSSA
- Customers
- Careers
- Partners
- Support

Follow Us



Terms & Conditions

Privacy Policy

Trust Center

© 2025 FOSSA Inc.